ML Model Deployment Strategies & Model Serving using FastAPI

Titanic: Machine Learning from Disaster | Complete Technical & Experimental Report

Student Name: Gowtham	**Registration Number: 231FA04094**
Assignment: MLOps CLA-4 (Deployment Strategies)	**Topic:** FastAPI Model Serving & Inference Modes
Dataset: Kaggle Titanic Disaster (train.csv)	**Target Variable:** Survived (0 = No, 1 = Yes)
Model Architecture: Scikit-Learn Pipeline (Random Forest)	**Evaluation Test Accuracy:** 80.45%
Model Serving Framework: FastAPI 0.143 + Uvicorn + Pydantic	**Artifacts:** model.pkl, batch_predictions.csv

1. System Architecture & Pipeline Design

Modern Machine Learning systems require seamless parity between training and serving environments. To eliminate **training-serving skew**, an end-to-end **Scikit-Learn Pipeline** was implemented combining data imputation, feature scaling, one-hot encoding, and a non-linear Random Forest Classifier into a unified serialized object (model.pkl). The model is trained once on an 80% stratified split (712 records) and evaluated on an unseen 20% test partition (179 records). This identical pipeline is strictly reused across Online, Batch, Streaming, and REST API serving modes.

Feature Group	Input Features	Preprocessing Pipeline Transformers
Numerical	Age, SibSp, Parch, Fare	SimpleImputer(strategy='median') → StandardScaler()
Categorical	Pclass, Sex, Embarked	SimpleImputer(strategy='most_frequent') → OneHotEncoder(handle_unknown='ignore')
Estimator	Cleaned Feature Vector	RandomForestClassifier(n_estimators=150, max_depth=6, random_state=42)

2. Task 1: Model Training & Evaluation Metrics

Evaluation on the held-out test partition demonstrated robust discrimination and balanced precision/recall: **Accuracy: 80.45%**, **F1-Score (Survived): 0.71**, **F1-Score (Non-survived): 0.85**.

Metric	Did Not Survive (Class 0)	Survived (Class 1)	Macro / Weighted Average
Precision	0.79	0.84	0.82 / 0.81
Recall	0.93	0.61	0.77 / 0.80
F1-Score	0.85	0.71	0.78 / 0.80

Support	110 records	69 records	Total: 179 unseen records

3. Demonstration of Inference Strategies (Tasks 2, 3, & 4)

Three distinct inference paradigms were implemented using the identical trained pipeline model.pkl:

Inference Strategy	Execution Mechanism	Performance Metrics	Primary Operational Advantage
Task 2: Online Inference online_inference.py	Synchronous request-response evaluation of 1 unseen record (Passenger #566, Mr. Alfred Davies).	Latency: **112.8 ms** Confidence: **88.19%** Result: Did Not Survive	Immediate instant feedback suitable for end-user web applications and real-time decision flows.
Task 3: Batch Inference batch_inference.py	Vectorized matrix transformation of 20 unseen records concurrently, exported to batch_predictions.csv.	Total Time: **133.8 ms** Per-Record: **6.69 ms** Throughput: **149.5 rec/s**	Extreme computational efficiency via vectorized CPU instructions and batched overhead amortisation.
Task 4: Streaming Inference streaming_inference.py	Simulated message stream of 10 incoming records arriving at 1.0s intervals, processed sequentially upon arrival.	Avg Model Latency: **36.4 ms** Inter-arrival: **1.0 sec** Events: 10/10 processed	Low-latency continuous inference for event-driven message architectures (Kafka, Kinesis, IoT feeds).

4. Task 5: Model Serving with FastAPI & Pydantic Validation

A production-grade REST API was engineered using **FastAPI** (app.py) running on ASGI server Uvicorn. Strict input validation and schema enforcement are executed via **Pydantic** models.

- **Endpoint GET /health:** Verifies service status, active loaded model, timestamp, and API version.
- **Endpoint POST /predict:** Accepts JSON passenger records, enforces field types, value constraints (e.g. $1 \leq$ Pclass ≤ 3, Age ≥ 0, Sex $\in$ {'male', 'female'}), passes inputs through the pipeline, and returns prediction class, label, probability, confidence, and inference latency.
- **Interactive Documentation:** Auto-generated OpenAPI / Swagger UI at /docs and ReDoc at /redoc.
- **Python Automated Client (client.py):** Automated test suite verifying health checks, valid predictions, and verifying that malformed payloads trigger HTTP 422 Unprocessable Entity with detailed error messages.

5. Task 6: Key Observations & Technical Answers

Question 1: How does Online Inference differ from Batch Inference?

Online Inference (also known as Real-Time or Request-Response Inference) operates synchronously: a client application sends an individual data instance (or small micro-batch) over a network protocol (HTTP/REST or gRPC) and expects an immediate low-latency prediction (typically sub-100ms). It prioritizes **latency and responsiveness**, powering interactive user-facing systems such as live fraud detection, interactive recommendation engines, and chatbot scoring.

In contrast, **Batch Inference** (Offline / Scheduled Inference) operates asynchronously on large volumes of data collected over time (e.g., millions of records processed hourly or daily). The entire batch is processed simultaneously using vectorized matrix transformations. Batch inference prioritizes **throughput, computational efficiency, and**

cost over turnaround time. In our benchmarks, batch processing reduced average per-record latency from **112.8 ms to 6.69 ms** (over a 16x improvement in throughput).

Question 2: What is the difference between Streaming Inference and Batch Inference?

Streaming Inference operates in near-real-time on continuous, unbounded event data streams (e.g., Apache Kafka, AWS Kinesis, IoT sensor feeds). As each event or small sliding micro-window arrives, the inference engine immediately computes and emits a prediction without waiting for data to be persisted to disk. Streaming inference is **event-driven, push-based, and handles dynamic temporal windows**.

Batch Inference, conversely, operates on static, bounded datasets persisted in data warehouses or object storage (e.g., Snowflake, BigQuery, AWS S3). Execution is typically **schedule-driven (via cron/Airflow) or threshold-driven**. While batch inference allows global dataset aggregations and high throughput, streaming inference enables real-time responsiveness to changing environmental conditions as events happen.

Question 3: Why is FastAPI used for model serving?

FastAPI has emerged as the premier standard for Python-based ML model serving due to several critical technical advantages:

1. **Asynchronous High Performance:** Built on Starlette and Uvicorn, FastAPI leverages Python's native async/await and ASGI concurrency, delivering throughput on par with NodeJS and Go, significantly outperforming legacy WSGI frameworks like Flask.
2. **Native Data Validation with Pydantic:** FastAPI uses Python type hints and Pydantic models to automatically validate incoming payloads, parse types, and return standardized HTTP 422 error details when invalid features are provided, protecting ML models from malformed inputs.
3. **Automatic OpenAPI & Swagger Documentation:** Interactive UI documentation (/docs and /redoc) is automatically generated from code annotations, enabling immediate testing and client integration without manual schema maintenance.
4. **Production Ergonomics:** Native dependency injection, built-in CORS middleware, lifespan events for lifecycle model loading, and seamless Docker containerization make FastAPI production-ready.

Question 4: Why should the same trained model be reused for all inference strategies?

Reusing the identical serialized model pipeline (model.pkl) across all deployment modes is a foundational MLOps best practice for four vital reasons:

1. **Prevention of Training-Serving Skew:** If preprocessing rules (e.g., median imputation values, scaling factors, one-hot category vocabularies) are reimplemented separately for online, batch, or streaming workflows, subtle mathematical divergence and code drift occur, leading to silent predictive degradation.
2. **Reproducibility & Consistency:** A given input passenger profile must yield the exact same prediction and probability score regardless of whether it was submitted via REST API, evaluated in a nightly batch CSV, or ingested from a streaming queue.
3. **Maintainability & Single Source of Truth:** Packaging feature transformers and model weights into one Scikit-Learn Pipeline allows teams to update, version, test, and audit a single model artifact rather than maintaining redundant preprocessing codebases.
4. **Operational Governance & Model Registry Parity:** In production MLOps pipelines (MLflow, Kubeflow, Vertex AI), tracking one versioned artifact hash ensures traceable governance, rollbacks, and compliance.

6. Comparative Matrix of Inference Deployment Strategies

Dimension	Online Inference	Batch Inference	Streaming Inference
Latency Profile	Ultra-low (Milliseconds, ~50-150ms)	High latency, high throughput (Minutes to Hours)	Near-real-time (Sub-second per event)
Data Input Format	Single JSON record or small payload	Bulk static datasets (CSV, Parquet, SQL tables)	Continuous event stream (Kafka, Kinesis, sockets)
Trigger Mechanism	Synchronous client request (HTTP/gRPC)	Cron scheduler, pipeline trigger, or queue threshold	Event arrival interrupt / message broker
Compute Strategy	On-demand persistent server instances (FastAPI)	Vectorized bulk processing / ephemeral worker clusters	Continuous stream processing engines (Flink, Spark)
Primary Metric	p95/p99 Latency & Availability	Records Processed / Sec & Cost Efficiency	Message Ingestion Lag & Stream Freshness

7. Submission Deliverables Verification

Deliverable File	Description & Verification Status
train.py	Task 1: Model training, preprocessing pipeline, evaluation (80.45%), exports model.pkl and unseen_test_data.csv.
online_inference.py	Task 2: Single-record online inference demonstration with latency measurement and confidence scores.
batch_inference.py	Task 3: Bulk inference over 20 unseen records; generates batch_predictions.csv.
streaming_inference.py	Task 4: Simulated event stream inference with 1-second arrival delay across 10 unseen records.
app.py	Task 5: FastAPI server with GET /health, POST /predict, and strict Pydantic input validation.
client.py	Task 5: Automated Python client testing /health, valid predictions, and verifying HTTP 422 for invalid payloads.
model.pkl	Serialized complete Scikit-Learn Pipeline combining imputation, scaling, one-hot encoding, and Random Forest classifier.
batch_predictions.csv	Generated output containing predictions, ground truth, and probabilities for 20 unseen passengers.
screenshots/	Visual proofs: Online, Batch, Streaming, Client Tests, and Swagger UI (/health, /predict valid, /predict invalid 422).

requirements.txt	Complete pinned dependencies required to reproduce and run the solution.
README.md	Full documentation, architecture overview, dataset links, execution commands, and observations.